

Nine tips for benchmarking Flink SQL on Confluent Cloud

We benchmarked one Flink SQL statement on a laptop and on Confluent Cloud, measuring how its throughput grew as we added compute. The SQL needed no changes, but several Confluent Cloud settings did.

Swipe for the nine tips →

TIP 1 OF 9

Make the job use the whole compute pool

Confluent's autoscaler decides how much of a pool a job uses. In a 20 CFU pool it used 10 and still reported everything as OK. Set the statement's `baseline_cfu` to the pool's size, and read it back to check.

20 CFU pool, left to the autoscaler: 10 used



With `baseline_cfu = 20`: all 20 used



TIP 2 OF 9

Choose a partition count that divides evenly by every pool size

Each worker reads a share of the topic's partitions. With 6 partitions and 4 workers, two workers get 2 and two get 1. The job runs at the pace of its busiest workers, so 4 workers do the work of 3.



6 partitions over 4 workers. Pick a count every pool size divides into, such as 40 for pools of 5, 10 and 20.

TIP 3 OF 9

Give the Kafka cluster enough capacity for your largest pool

A Flink benchmark needs Kafka to keep up, or it measures Kafka instead. Confluent measures Kafka capacity in eCKU; if you cap it to control cost, set the cap for your largest test.

job, doubled

could do 120

Kafka limit

100

you measure

100, not 120

Records a second, made-up numbers. The job looks as if it scales worse than it does.

TIP 4 OF 9

Count your test data by reading it, not from the topic's size

A topic's size also includes records whose write was cancelled, and Flink skips those. A benchmark needs to know exactly how many records it starts with.

A loading job writes 1,000 records and is stopped before the last 400 are committed

topic size

600

400 cancelled

Flink reads

600

TIP 5 OF 9

Check your results with a snapshot query

After a benchmark you count records to make sure nothing was lost. A normal Flink SQL count sends a new running total for every record, and you have to read through all of them to reach the final number.

NORMAL COUNT

Counting 100 million records sends 100 million updates.

Hours

SNAPSHOT QUERY

Sends only the final total.

About a minute

Set `sql.snapshot.mode = now`

TIP 6 OF 9

Create a new compute pool for each size you test

Confluent won't let you lower a pool's maximum size, so to test a smaller size after a larger one you need a new pool. We used a new pool for every size we tested.

NOT ALLOWED

Lower one pool from 20 CFU to 10

INSTEAD

A new 10 CFU pool

TIP 7 OF 9

Wait for the usage figures before you measure

- Create the pool and start the job
- Usage figures appear
a few minutes later
- Figures keep coming
about three minutes behind
- Start measuring
once they show the job using the whole pool

TIP 8 OF 9

Make sure a benchmark deletes what it creates, even if it crashes

A test that stops halfway can leave a cluster running, and billing, for hours.

CLEANUP AT THE END

A crash skips it. The cluster keeps running.

CLEANUP ON EVERY EXIT

Runs whether the test passes, stops or crashes.

TIP 9 OF 9

Allow for the cost report's delay

Confluent's daily cost list runs about a day behind, so today's spend isn't in it yet.

yesterday

in the cost list

today

not there yet

FLINK SQL • CONFLUENT CLOUD

All nine are built into our open-source skill

It measures whether a Flink pipeline scales, and now runs Flink SQL on a laptop and on Confluent Cloud.

github.com/jimzucker/scalable-flink-skill